

Gatling API Tool: Detailed Scenario Runbook

Scenario: **orders-service / qa**

Flow: **GET /orders -> GET /orders/{orderId} -> POST /orders**

Targets: **Success >= 99%, p95 <= 1200ms, p99 <= 2000ms**

1. Installation and Startup

- Run `check-prerequisites.bat`
- Run `start-ui-workspace.bat`
- Verify UI Runner API is `http://127.0.0.1:8787`

```
cd gatling-api-tool
check-prerequisites.bat
start-ui-workspace.bat
```

Run Setup

Run Setup always shows generated YAML and Scala output. In Expert / Raw mode you can override the generated YAML directly for download and real Gatling execution.

Saved Regression Suites

Save the current workspace as a reusable regression suite, reload it later, or execute a previously saved suite directly from this page.

SUITE NAME

nightly-regression

SAVED SUITES

No saved suite selected

Save Current Suite

Load Selected Suite

Run Saved Preview

Run Saved Gatling

Delete Selected Suite

Saved: orders-qa-regression | Apps: 1 | Scenarios: 3 | Mode: expert

UI RUNNER API

http://127.0.0.1:8787

Start Runner

Check Runner

Run Real Load (Gatling)

GATLING REPORT UI LINK

http://... or C:\path\index.html

Open Gatling Report

Runner status demo: ready for real run.

Preview runner supports 'url', 'queryParams', and 'formParams'. Env-backed auth, multipart uploads, and local 'bodyFile' paths may differ from backend execution.

Generated Gatling YAML

This is the canonical config produced from the structured builder. It is the configuration used by backend execution, YAML download, and Expert raw-mode override sync.

```
applications:
  "api":
    enabled: true
    service:
      baseUrl: "https://jsonplaceholder.typicode.com"
    defaultHeaders:
      "Accept": "application/json"
      "Content-Type": "application/json"
    activeEnvironment: "env1"
  environments:
    "env1":
      enabled: true
  injectionProfiles:
    "default_ramp":
      injectionType: "rampUsers"
      users: 10
      rampDurationSec: 30
      "hour_60k":
```

2. App and Base Service Setup

- Application: orders-service
- Base URL: https://qa.api.company.com
- Auth: bearer via env API_TOKEN
- Assertions: minSuccess 99, maxRT 2000, p95 1200

PreviousNext

Step 1 of 4: Apps & Service

Gatling API Performance UI

Guided setup for multi-app, multi-environment API load testing.

Step 1: Add Applications
Create each app once and provide its base URL in the Applications table.

Step 2: Define Load
Create Injection Profiles, then map every scenario to one profile.

Step 3: Run
Use Run Concurrent Hits (preview) or Run Real Load (actual Gatling).

Service Baseline + Assertions

This baseline applies to active app and is merged with selected environment overrides.

BASE URL	AUTH TYPE	BEARER TOKEN ENV	BASIC USERNAME ENV	BASIC PASSWORD ENV
https://qa.api.company.com	none	API_TOKEN	API_USER	API_PASSWORD
HEADER NAME	HEADER VALUE ENV	UI ITERATIONS PER SCENARIO	MIN SUCCESS %	MIN REQUESTS/SEC
x-api-key	API_KEY	3	99	0
MAX RT (MS)	P90 RT (MS)	P95 RT (MS)	P99 RT (MS)	MAX FAILED REQUESTS
2000	0	1200	0	0

'UI Iterations' is only for browser preview run. Gatling backend run uses injection profile settings.

Applications

Set each app base URL once here. Switch Active app to configure scenarios/environments under that app.

Enabled	Active	App Name	Base URL (Set Once)	Action
☒	☐	app1	https://jsonplaceholder.typicode.com	Remove

Add Application

1. Apps & Service2. Load & Scenarios3. Run Setup4. Reports

PreviousNext

Step 1 of 4: Apps & Service

Gatling API Performance UI

Guided setup for multi-app, multi-environment API load testing.

Step 1: Add Applications
Create each app once and provide its base URL in the Applications table.

Step 2: Define Load
Create Injection Profiles, then map every scenario to one profile.

Step 3: Run
Use Run Concurrent Hits (preview) or Run Real Load (actual Gatling).

Service Baseline + Assertions

This baseline applies to active app and is merged with selected environment overrides.

BASE URL	AUTH TYPE	BEARER TOKEN ENV	BASIC USERNAME ENV	BASIC PASSWORD ENV
https://qa.api.company.com	none	API_TOKEN	API_USER	API_PASSWORD
HEADER NAME	HEADER VALUE ENV	UI ITERATIONS PER SCENARIO	MIN SUCCESS %	MIN REQUESTS/SEC
x-api-key	API_KEY	3	99	0
MAX RT (MS)	P90 RT (MS)	P95 RT (MS)	P99 RT (MS)	MAX FAILED REQUESTS
2000	0	1200	0	0

'UI Iterations' is only for browser preview run. Gatling backend run uses injection profile settings.

Applications

Set each app base URL once here. Switch Active app to configure scenarios/environments under that app.

Enabled	Active	App Name	Base URL (Set Once)	Action
☒	☐	app1	https://jsonplaceholder.typicode.com	Remove

3. Injection Profiles

- smoke_5: 5 users / 15 sec
- baseline_20: ramp 20 users / 60 sec
- stress_80: ramp 80 users / 180 sec
- Map scenario to baseline_20 for QA baseline run

[illegible]

4. Scenario Steps (Core)

- Step 1: GET /orders expected 200
- Step 2: GET /orders/#{orderId} expected 200
- Step 3: POST /orders expected 201

The screenshot displays the 'Gatling API Performance UI' interface. At the top, a header bar includes 'ENTERPRISE GATLING WORKSPACE' and a title 'Gatling API Performance UI'. Below the title, a subtitle reads: 'Configure environments, build scenario-driven API workloads, generate Gatling assets, and review real execution results from a single operational workspace.' To the right of the title, two boxes are present: 'CONFIGURATION MODEL' with 'Multi-App' (described as 'Shared service baselines, reusable injection profiles, and environment-specific overrides.') and 'EXECUTION MODES' with 'Preview + Real Run' (described as 'Use browser preview for rapid checks and backend Gatling runs for actual performance reports.').

The main content area is titled 'Test Flow' and includes a subtitle: 'Move through the workflow in order: establish service defaults, compose scenarios, generate executable artifacts, and review resulting performance evidence.' Below this, a progress bar shows four steps: '1. Apps & Service', '2. Load & Scenarios' (the current step), '3. Run Setup', and '4. Reports'. Navigation buttons 'Previous' and 'Next' are located below the progress bar, with 'Next' being the active button. A small text below the buttons indicates 'Step 2 of 4: Load & Scenarios'.

Below the 'Test Flow' section, the interface shows 'Editing details for: app1' and a status message 'Validation passed. Configuration looks ready.' This is followed by a 'WORKSPACE MODE' section with three tabs: 'Basic' (selected), 'Advanced', and 'Expert / Raw'. A note states: 'Basic mode keeps the workflow narrow: essential service fields, simple scenarios, and generated artifacts without advanced flow logic.' Below the workspace mode tabs, there are three more tabs: 'Scenarios' (selected), 'Injection Profiles', and 'Environments'.

The 'Scenarios' section has a subtitle: 'Start with one scenario and one API step. Set the path, expected status, and an injection profile. Move to Advanced when you need branching, captures, or request-level tuning.' Below this, a small text block provides additional context: 'Each scenario should contain at least one API step and one Injection Profile mapping. API cards are compact by default. Add optional items like headers, query params, form params, checks, captures, and uploads only when needed. New apps start with a 'jsonplaceholder.typicode.com' sample scenario so you can verify YAML and Scala script generation immediately.'

5. Checks and Captures

- Add checks: `jsonPathExists($.items[0].id)`, `bodyContains("order")`
- Capture: `$.items[0].id -> orderId`
- Use capture in next step path: `/orders/#{orderId}`

The screenshot displays a REST client interface with two API configurations, API 1 and API 2, each with a 'Remove API' button in the top right corner.

API 1 Configuration:

- FEEDER TYPE:** none
- FEEDER FILE:** data/users.csv
- FEEDER MODE:** queue
- API Name:** Get Posts
- METHOD:** GET
- PATH:** /posts
- ABSOLUTE URL:** optional override
- EXPECTED STATUS:** 200
- RETRY COUNT:** 0
- PAUSE (MS):** 0
- Checks:** jsonPathExists(\$.items[0].id), bodyContains("order")
- Capture:** \$.items[0].id -> orderId

API 2 Configuration:

- API Name:** Get Post By Id
- METHOD:** GET
- PATH:** /posts/1
- ABSOLUTE URL:** optional override
- EXPECTED STATUS:** 200
- RETRY COUNT:** 0
- PAUSE (MS):** 0
- Checks:** jsonPathExists(\$.items[0].id), bodyContains("order")
- Capture:** \$.items[0].id -> orderId

6. Environments, Headers, Certs

- Define environment qa with specific URL/auth overrides
- Use global headers in Advanced mode
- Configure TLS cert/trust store only when required

ENTERPRISE GATLING WORKSPACE

Gatling API Performance UI

Configure environments, build scenario-driven API workloads, generate Gatling assets, and review real execution results from a single operational workspace.

CONFIGURATION MODEL

Multi-App
Shared service baselines, reusable injection profiles, and environment-specific overrides.

EXECUTION MODES

Preview + Real Run
Use browser preview for rapid checks and backend Gatling runs for actual performance reports.

Test Flow

Move through the workflow in order: establish service defaults, compose scenarios, generate executable artifacts, and review resulting performance evidence.

1. Apps & Service

2. Load & Scenarios

3. Run Setup

4. Reports

Previous

Next

Step 2 of 4: Load & Scenarios

Editing details for: app1

Validation passed. Configuration looks ready.

WORKSPACE MODE

Basic

Advanced

Expert / Raw

Advanced mode exposes the full structured builder: headers, certificates, request tuning, captures, branching, and richer scenario controls.

Scenarios

Injection Profiles

Environments

Headers

Certificates

Named Environments

Define environment-specific overrides and select which ones to execute in Run Concurrent Hits.

Enabled	Active	Name	Base URL	Auth Type	Auth Param 1	Auth Param 2	Header Name	Headers JSON	TLS Enabled	KeyStore	TrustStore	Action
☒	☐	env1		r	API_TOKEN		x-api-key	{}				Remove

ENTERPRISE GATLING WORKSPACE

Gatling API Performance UI

Configure environments, build scenario-driven API workloads, generate Gatling assets, and review real execution results from a single operational workspace.

CONFIGURATION MODEL

Multi-App
Shared service baselines, reusable injection profiles, and environment-specific overrides.

EXECUTION MODES

Preview + Real Run
Use browser preview for rapid checks and backend Gatling runs for actual performance reports.

Test Flow

Move through the workflow in order: establish service defaults, compose scenarios, generate executable artifacts, and review resulting performance evidence.

1. Apps & Service

2. Load & Scenarios

3. Run Setup

4. Reports

Previous

Next

Step 2 of 4: Load & Scenarios

Editing details for: app1

Validation passed. Configuration looks ready.

WORKSPACE MODE

Basic

Advanced

Expert / Raw

Advanced mode exposes the full structured builder: headers, certificates, request tuning, captures, branching, and richer scenario controls.

Scenarios

Injection Profiles

Environments

Headers

Certificates

TLS Certificates

Use this only when target APIs require client cert/truststore or custom TLS behavior.

ENABLE TLS

false

KEYSTORE PATH

KEYSTORE TYPE

PKCS12

KEYSTORE PASSWORD ENV

KEYSTORE_PASSWORD

TRUSTSTORE PATH

TRUSTSTORE TYPE

TRUSTSTORE PASSWORD ENV

SKIP TLS VERIFY

false

7. Expert / Raw YAML

- Click **Sync Generated YAML Into Editor**
- Enable **Use Raw YAML: On** only for edge-case overrides
- Keep diff minimal and preserve profile/assertion structure

ENTERPRISE GATLING WORKSPACE

Gatling API Performance UI

Configure environments, build scenario-driven API workloads, generate Gatling assets, and review real execution results from a single operational workspace.

CONFIGURATION MODEL
Multi-App
Shared service baselines, reusable injection profiles, and environment-specific overrides.

EXECUTION MODES
Preview + Real Run
Use browser preview for rapid checks and backend Gatling runs for actual performance reports.

Test Flow

Move through the workflow in order: establish service defaults, compose scenarios, generate executable artifacts, and review resulting performance evidence.

1. Apps & Service

2. Load & Scenarios

3. Run Setup

4. Reports

Previous

Next

Step 3 of 4: Run Setup

Run Setup

Run Setup always shows generated YAML and Scala output. In Expert / Raw mode you can override the generated YAML directly for download and real Gatling execution.

Saved Regression Suites

Save the current workspace as a reusable regression suite, reload it later, or execute a previously saved suite directly from this page.

SUITE NAME

nightly-regression

SAVED SUITES

No saved suite selected

Save Current Suite

Load Selected Suite

Run Saved Preview

Run Saved Gatling

Delete Selected Suite

Saved: orders-qa-regression | Apps: 1 | Scenarios: 3 | Mode: expert

UI RUNNER API

GATLING REPORT UI LINK

```
setup(
  app1_Read_Posts_scenario.injectOpen(rampUsers(10) during (30.seconds)).protocols(http_app1)
)
.assertions(
  details("app1 :: Read Posts").successfulRequests.percent.gte(99),
  details("app1 :: Read Posts").responseTime.max.lte(2000),
  details("app1 :: Read Posts").responseTime.percentile3.lte(1200)
)

private def requireEnv(name: String): String = {
  val value = sys.env.getOrElse(name, "")
  require(value.nonEmpty, s"Missing env var: $name")
  value
}

private def basicAuthHeader(userEnv: String, passEnv: String): String = {
  val raw = s"${requireEnv(userEnv)}:${requireEnv(passEnv)}"
  "Basic " + Base64.getEncoder.encodeToString(raw.getBytes(StandardCharsets.UTF_8))
}

private def readFile(path: String): String = Files.readString(Paths.get(path))
}
```

Expert / Raw YAML Override

Use this only when you need a full-fidelity YAML escape hatch beyond the structured builder. When enabled, real runs and YAML download use this editor content directly. Scala script generation still comes from the structured UI model.

Sync Generated YAML Into Editor

Use Raw YAML: Off

```
applications:
  "app1":
    enabled: true
    service:
      baseUrl: "https://qa.api.company.com"
      defaultHeaders:
        "Accept": "application/json"
        "Content-Type": "application/json"
    activeEnvironment: "env1"
  environments:
    "env1":
      enabled: true
      injectionProfiles:
        "default_ramp":
          injectionType: "rampUsers"
          users: 10
          rampDurationSec: 30
```

7A. Conditional Statements (If/Else) Example

- Drive branching using captured/session values like `#{customerTier}`
- If premium: call premium route; else call standard route
- Use explicit expected status on both branch and else path

steps:

```
- name: "Route By Tier"
  method: "GET"
  path: "/orders"
  branches:
    - name: "Premium Path"
      when:
        variable: "customerTier"
        operator: "equals"
        value: "premium"
        method: "GET"
        path: "/orders/premium"
        expectedStatus: 200
      elseMethod: "GET"
      elsePath: "/orders/standard"
      elseExpectedStatus: 200
```

ENTERPRISE GATLING WORKSPACE

Gatling API Performance UI

Configure environments, build scenario-driven API workloads, generate Gatling assets, and review real execution results from a single operational workspace.

CONFIGURATION MODEL

Multi-App
Shared service baselines, reusable injection profiles, and environment-specific overrides.

EXECUTION MODES

Preview + Real Run
Use browser preview for rapid checks and backend Gatling runs for actual performance reports.

Test Flow

Move through the workflow in order: establish service defaults, compose scenarios, generate executable artifacts, and review resulting performance evidence.

1. Apps & Service

2. Load & Scenarios

3. Run Setup

4. Reports

Previous

Next

Step 2 of 4: Load & Scenarios

Editing details for: app1

Validation passed. Configuration looks ready.

WORKSPACE MODE

Basic

Advanced

Expert / Raw

Advanced mode exposes the full structured builder: headers, certificates, request tuning, captures, branching, and richer scenario controls.

Scenarios

Injection Profiles

Environments

Headers

Certificates

TLS Certificates

Use this only when target APIs require client cert/truststore or custom TLS behavior.

ENABLE TLS	KEYSTORE PATH	KEYSTORE TYPE	KEYSTORE PASSWORD ENV	TRUSTSTORE PATH
false		PKCS12	KEYSTORE_PASSWORD	
TRUSTSTORE TYPE	TRUSTSTORE PASSWORD ENV	SKIP TLS VERIFY		
PKCS12	TRUSTSTORE_PASSWORD	false		

8. Real Run and Diagnostics

- Run **Check Runner**, then **Run Real Load (Gatling)**
- Track run state, job id, and output tail
- Confirm report availability is Final

Run Setup

Run Setup always shows generated YAML and Scala output. In Expert / Raw mode you can override the generated YAML directly for download and real Gatling execution.

Saved Regression Suites

Save the current workspace as a reusable regression suite, reload it later, or execute a previously saved suite directly from this page.

SUITE NAME

nightly-regression

SAVED SUITES

No saved suite selected

Save Current Suite

Load Selected Suite

Run Saved Preview

Run Saved Gatling

Delete Selected Suite

Saved: orders-qa-regression | Apps: 1 | Scenarios: 3 | Mode: expert

UI RUNNER API

http://127.0.0.1:8787

Start Runner

Check Runner

Run Real Load (Gatling)

GATLING REPORT UI LINK

http://... or C:\path\index.html

Open Gatling Report

Runner status demo: ready for real run.

Preview runner supports 'url', 'queryParams', and 'formParams'. Env-backed auth, multipart uploads, and local 'bodyFile' paths may differ from backend execution.

Generated Gatling YAML

This is the canonical config produced from the structured builder. It is the configuration used by backend execution, YAML download, and Expert raw-mode override sync.

```
applications:
  appl1:
    enabled: true
    service:
      baseUrl: "https://jsonplaceholder.typicode.com"
    defaultHeaders:
      "Accept": "application/json"
      "Content-Type": "application/json"
    activeEnvironment: "env1"
  environments:
    env1:
      enabled: true
  injectionProfiles:
    "default_ramp":
      injectionType: "rampUsers"
      users: 10
      rampDurationSec: 30
    "hour_60k":
      injectionType: "pacedUsers"
      users: 10
```

9. Report Analysis

- Primary KPIs: success %, p95, p99, max RT
- Use failure reason table first for triage
- Use parity diagnostics to distinguish preview limitations

ENTERPRISE GATLING WORKSPACE

Gatling API Performance UI

Configure environments, build scenario-driven API workloads, generate Gatling assets, and review real execution results from a single operational workspace.

CONFIGURATION MODEL

Multi-App
Shared service baselines, reusable injection profiles, and environment-specific overrides.

EXECUTION MODES

Preview + Real Run
Use browser preview for rapid checks and backend Gatling runs for actual performance reports.

Test Flow

Move through the workflow in order: establish service defaults, compose scenarios, generate executable artifacts, and review resulting performance evidence.

1. Apps & Service

2. Load & Scenarios

3. Run Setup

4. Reports

Previous

Next

Step 4 of 4: Reports

Run Results

No active run. Start a test to see live execution status and report updates here.

TEST STATUS		CURRENT RUN ID		BACKEND JOB ID		REPORT AVAILABILITY	
Idle		-		-		Pending	
SCENARIOS	API STEPS	TOTAL REQUESTS	SUCCESS	FAILURE	SUCCESS %	MIN RT	AVG RT
0	0	0	0	0	0.00%	0 ms	0 ms
P95 RT	P99 RT	MAX RT	PARITY WARNINGS				
0 ms	0 ms	0 ms	0				

Per API Step

10. Save Suite and Release

- Save suite as `orders-qa-regression`
- Create release package via `build-release.bat`
- Share zip + `latest-bundle.sha256`

`build-release.bat`

Move through the workflow in order: establish service defaults, compose scenarios, generate executable artifacts, and review resulting performance evidence.

1. Apps & Service 2. Load & Scenarios 3. Run Setup 4. Reports

Previous Next

Step 3 of 4: Run Setup

Run Setup

Run Setup always shows generated YAML and Scala output. In Expert / Raw mode you can override the generated YAML directly for download and real Gatling execution.

Saved Regression Suites

Save the current workspace as a reusable regression suite, reload it later, or execute a previously saved suite directly from this page.

SUITE NAME:

SAVED SUITES:

Save Current Suite Load Selected Suite Run Saved Preview Run Saved Gatling Delete Selected Suite

Saved: orders-qa-regression | Apps: 1 | Scenarios: 3 | Mode: expert

UI RUNNER API: Start Runner Check Runner Run Real Load (Gatling) GATLING REPORT UI LINK: Open Gatling Report

Runner status demo: ready for real run.
Preview runner supports 'url', 'queryParams', and 'formParams'. Env-backed auth, multipart uploads, and local 'bodyFile' paths may differ from backend execution.

Generated Gatling YAML

This is the canonical config produced from the structured builder. It is the configuration used by backend execution, YAML download, and Expert raw-mode override sync.

```
applications:
  "appl1":
    enabled: true
  services:
    baseUrl: "https://jsonplaceholder.typicode.com"
  defaultHeaders:
    "Accept": "application/json"
```

Generated automatically from the current workspace UI captures.